

GLOSSÁRIO DE INTELIGÊNCIA ARTIFICIAL E LLMs

Engenharia de Computação 2026

Autores:

Vitória Aparecida Vendausen

Matheus Dapper

Compêndio acadêmico, formal e didático sobre arquiteturas de Large Language Models (LLMs): fundamentos teóricos de IA, pipelines de tokenização e embeddings vetoriais, e implementação profunda de mecanismos de atenção (Self-Attention, Causal Attention e Multi-Head Attention).

Agosto 2026

Sumário

1	Capítulo 1: Fundamentos de Inteligência Artificial e Modelos de Base	2
2	Capítulo 2: Tokenização e Embeddings	11
3	Capítulo 3: Mecanismos de Atenção (Coding Attention Mechanisms)	19

Capítulo 1: Fundamentos de Inteligência Artificial e Modelos de Base

Reinforcement Learning from Human Feedback (RLHF)

Português: Aprendizado por reforço com feedback humano (substantivo masculino)

- **Definição:** Esse tipo de otimização funciona com especialistas humanos avaliando as respostas do modelo para demonstrar o que é um conteúdo seguro, verdadeiro e útil.
- **Função no Modelo:** O sistema propõe um texto para que um treinador humano julgue ou escolha a melhor resposta para recompensar o algoritmo, guiando o modelo para longe de boatos, preconceitos ou piadas indesejadas.
- **Relação com Outros Conceitos:** Geralmente é o método utilizado para realizar o alinhamento do modelo após a conclusão da etapa de ajuste fino.
- **Exemplo:** O ChatGPT propõe três textos possíveis para o usuário e um revisor humano pontua a melhor opção, fazendo com que o sistema aprenda a priorizá-la no futuro.

Supervised Learning

Português: Aprendizado supervisionado (substantivo masculino)

- **Definição:** O paradigma tradicional de treinamento no qual um algoritmo aprende comparando suas palpites a um conjunto de respostas fornecido junto com os dados de treino.
- **Função no Modelo:** Ao analisar milhares de exemplos pareados, o sistema aprende a executar uma ação sobre um texto para obter outro texto desejado.
- **Relação com Outros Conceitos:** Serve como base para modelos de instrução direta ou etapas de ajuste fino.
- **Exemplo:** Mostrar ao algoritmo quais e-mails foram classificados por pessoas como “Spam” ou “Caixa de Entrada”, permitindo que ele aprenda a executar essa mesma tarefa sozinho.

Self-Supervised Learning

Português: Aprendizado auto-supervisionado (substantivo masculino)

- **Definição:** Um conjunto de métodos em que o algoritmo se treina em dados não rotulados, dispensando a necessidade de anotações humanas.
- **Função no Modelo:** Permite que o sistema analise milhares de páginas da web para aprender conceitos gerais de linguagem por conta própria.
- **Relação com Outros Conceitos:** A técnica fundamental que constitui toda a etapa de pré-treinamento.
- **Exemplo:** Pedir para o modelo adivinhar a palavra “azul” na frase “O céu está muito _____ hoje”, utilizando o restante da internet como dado de treinamento.

Fine-Tuning

Português: Ajuste fino (substantivo masculino)

- **Definição:** O ajuste dos pesos de um modelo para especializá-lo em uma tarefa específica, expondo-o a um conjunto mais restrito de informações.
- **Função no Modelo:** Permite atualizar um modelo de linguagem genérico e pré-treinado para torná-lo melhor em uma tarefa específica sem desperdiçar recursos.
- **Relação com Outros Conceitos:** Geralmente posicionado entre o pré-treinamento e a implantação final, funcionando como um caminho para criar ferramentas úteis a partir de modelos grandes e genéricos.
- **Exemplo:** Expor uma IA de uso geral a milhares de documentos jurídicos para capacitá-la a redigir peças do Direito.

Alignment

Português: Alinhamento (substantivo masculino)

- **Definição:** O ajuste do comportamento do modelo para atender às expectativas humanas, evitando a geração de conteúdos prejudiciais.
- **Função no Modelo:** Filtra as respostas do sistema para impedir que informações tóxicas, perigosas ou tabus sejam geradas.
- **Relação com Outros Conceitos:** Geralmente realizado por meio do aprendizado por reforço com feedback humano (RLHF) para moldar a personalidade de um assistente virtual.
- **Exemplo:** Recompensar o modelo por se recusar a responder “Como invadir um sistema” e penalizá-lo caso ele forneça dicas úteis de invasão.

Hallucination

Português: Alucinação (substantivo masculino)

- **Definição:** A produção de informações não encontradas nos dados de treinamento, frequentemente apresentadas como fatos ou citações reais.
- **Função no Modelo:** Um risco inerente ao sistema de predição da próxima palavra, que faz a IA dar palpites além das informações concretas que possui.
- **Relação com Outros Conceitos:** Frequentemente ligada a configurações de alta temperatura e à tarefa de predição da próxima palavra.
- **Exemplo:** Pedir o resumo de um livro que nunca foi publicado e receber uma resposta longa, com direito a citações fabricadas e uma data de lançamento falsa.

Transformer Architecture

Português: Arquitetura Transformer (substantivo feminino)

- **Definição:** Um tipo de rede neural que processa textos analisando todas as palavras simultaneamente, ao contrário do processamento sequencial antigo.
- **Função no Modelo:** Permite realizar cálculos em grandes volumes de texto de uma só vez e conectar ideias separadas por vários parágrafos.
- **Relação com Outros Conceitos:** Substituiu as arquiteturas recorrentes antigas e viabilizou os modelos de base modernos.

- **Exemplo:** A tecnologia fundamental por trás da maioria dos assistentes modernos, como ChatGPT, Gemini ou Claude.

Selective Attention

Português: Atenção seletiva (substantivo feminino)

- **Definição:** O mecanismo pelo qual a rede neural foca mais em palavras específicas para compreender o seu contexto real.
- **Função no Modelo:** Permite ao modelo associar determinadas expressões para descartar significados irrelevantes em um dado contexto.
- **Relação com Outros Conceitos:** Geralmente implementada através do mecanismo de auto-atenção.
- **Exemplo:** Diante da frase “O banco da praça estava molhado”, o sistema entende que a palavra “banco” se refere ao assento, descartando o sentido de instituição financeira.

Autoregression

Português: Autoregressão (substantivo feminino)

- **Definição:** Uma técnica que permite ao modelo escrever textos prevendo uma palavra de cada vez.
- **Função no Modelo:** Auxilia o modelo a gerar textos longos e coerentes, utilizando a última palavra digitada como ponto de partida para a próxima.
- **Relação com Outros Conceitos:** Habilita o comportamento de predição da próxima palavra na parte decodificadora do modelo.
- **Exemplo:** No momento em que o sistema digita “O”, ele calcula que “sol” é mais provável que “céu” e, após “O sol”, projeta o verbo “brilha”.

Emergent Behavior

Português: Capacidade emergente (substantivo feminino)

- **Definição:** O surgimento de novas habilidades e competências que não foram treinadas de forma explícita.
- **Função no Modelo:** Torna o sistema capaz de realizar tarefas surpreendentes que não lhe foram ensinadas diretamente.
- **Relação com Outros Conceitos:** Geralmente surge em modelos de linguagem de grande escala devido ao aumento no volume de dados e na quantidade de parâmetros.
- **Exemplo:** Um modelo de linguagem projetado para prever a próxima palavra conseguir resolver problemas matemáticos complexos e traduzir poesias antigas.

Encoder

Português: Codificador (substantivo masculino)

- **Definição:** A parte da rede neural cujo propósito é compreender o contexto e o significado de um determinado texto.
- **Função no Modelo:** Ideal para tarefas que exigem análise ou compreensão profunda, em vez da escrita criativa.
- **Relação com Outros Conceitos:** Geralmente combinado com a parte decodificadora em modelos do tipo Codificador-Decodificador (Encoder-Decoder).
- **Exemplo:** Um sistema automatizado que lê avaliações de clientes e as separa em grupos de comentários positivos ou negativos.

Zero-Shot Learning

Português: Comportamento de nenhuma amostra (substantivo masculino)

- **Definição:** A capacidade de realizar uma tarefa sem ter sido explicitamente treinado para ela.
- **Função no Modelo:** Permite que o sistema siga instruções que não faziam parte dos seus dados originais de treinamento.
- **Relação com Outros Conceitos:** Pode ser visto como uma forma de capacidade emergente.
- **Exemplo:** Pedir para escrever um poema sobre o mar no estilo de Fernando Pessoa e obter uma resposta válida sem fornecer nenhum contexto adicional.

Few-Shot Learning

Português: Comportamento de poucas amostras (substantivo masculino)

- **Definição:** A capacidade de deduzir regras gerais para uma tarefa a partir de poucos exemplos incluídos junto com a própria requisição.
- **Função no Modelo:** Permite ao sistema adaptar seu comportamento a diferentes tarefas sem a necessidade de treinamentos adicionais.
- **Relação com Outros Conceitos:** Funciona como um meio-termo entre o aprendizado sem amostras (zero-shot) e o ajuste fino (fine-tuning).
- **Exemplo:** Pedir para converter frases em gírias fornecendo alguns exemplos no comando e obter resultados corretos para os pedidos seguintes.

Decoder

Português: Decodificador (substantivo masculino)

- **Definição:** A parte da rede neural focada na geração de textos fluídos e coerentes.
- **Função no Modelo:** Permite ao sistema continuar escrevendo ao analisar o contexto e prever as próximas palavras mais prováveis.
- **Relação com Outros Conceitos:** Geralmente aparece ao lado do codificador ou é utilizado de forma autônoma em modelos autorregressivos.
- **Exemplo:** O componente responsável por fazer o ChatGPT continuar gerando novo texto de forma sequencial na sua tela.

Embeddings

Português: Vetores de incorporação / Embeddings (substantivo masculino plural)

- **Definição:** A associação de palavras ou conceitos a coordenadas numéricas dentro de um espaço multidimensional.
- **Função no Modelo:** Permite comparar o grau de parentesco e o significado entre diferentes palavras ou frases.
- **Relação com Outros Conceitos:** Geralmente calculado pela etapa codificadora do modelo.
- **Exemplo:** O sistema entender que a palavra “rainha” está mais próxima de “rei” do que de “moeda” ao analisar suas coordenadas numéricas.

Manual Feature Extraction

Português: Extração manual de características (substantivo feminino)

- **Definição:** Formato antigo de programação onde especialistas humanos precisavam definir manualmente o que o computador deveria procurar.
- **Função no Modelo:** Útil para tarefas muito específicas, porém limitado pelo conhecimento do seu criador.
- **Relação com Outros Conceitos:** O método fundamental que foi gradualmente substituído pelo aprendizado auto-supervisionado.
- **Exemplo:** Escrever regras de código estritas determinando como classificar e-mails recebidos como spam ou não.

Prompt

Português: Instrução de entrada / Prompt (substantivo feminino)

- **Definição:** Um comando ou pergunta feita à IA para que ela execute uma tarefa específica.
- **Função no Modelo:** A instrução direta que o usuário digita para descrever o que deseja que a IA faça.
- **Relação com Outros Conceitos:** Pode ser utilizado para definir exemplos de poucas amostras (few-shot) ou comandos diretos sem amostras (zero-shot).
- **Exemplo:** Pedir para resumir a Revolução Francesa em três tópicos simples para uma criança.

Generative Artificial Intelligence (GenAI)

Português: Inteligência artificial generativa (substantivo feminino)

- **Definição:** Sistemas modernos de IA capazes de gerar novos textos, áudios, imagens ou códigos de computador através da análise de padrões existentes.
- **Função no Modelo:** Permite ao sistema ir além das tarefas puramente linguísticas e entrar na criação ativa de mídias.
- **Relação com Outros Conceitos:** Os modelos de base são geralmente fundamentados em tecnologias de IA generativa.

- **Exemplo:** Pedir para criar a imagem de um astronauta andando a cavalo em Marte e receber uma pintura detalhada em resposta.

Context Window

Português: Janela de contexto (substantivo feminino)

- **Definição:** A quantidade de texto que o modelo consegue analisar de uma só vez para entender o que o usuário está solicitando.
- **Função no Modelo:** Determina o volume máximo de texto que pode ser processado ou lembrado em um dado momento.
- **Relação com Outros Conceitos:** Impõe o limite operacional para que o mecanismo de autoatenção consiga acompanhar uma conversa longa.
- **Exemplo:** A capacidade de colar um livro inteiro no chat para que o modelo analise e responda a perguntas sobre ele.

Self-Attention Mechanism

Português: Mecanismo de autoatenção (substantivo masculino)

- **Definição:** O processo pelo qual o modelo compara cada palavra de uma frase com todas as outras palavras da mesma sentença.
- **Função no Modelo:** Mantém o rastreo de todos os termos utilizados para que a máquina não se esqueça das partes iniciais de uma frase longa.
- **Relação com Outros Conceitos:** O mecanismo fundamental que permite à arquitetura Transformer processar frases inteiras de uma só vez.
- **Exemplo:** Na frase “A menina que morava na casa amarela no fim da rua comprou um livro”, o sistema associa a primeira palavra “A” (referente à menina) diretamente ao verbo “comprou”.

Foundation Model

Português: Modelo de base (substantivo masculino)

- **Definição:** Sistemas de IA de altíssimo porte que fornecem uma compreensão ampla e de uso geral sobre o mundo.
- **Função no Modelo:** Atua como a fundação primária para a maioria das aplicações práticas de IA generativa.
- **Relação com Outros Conceitos:** Geralmente passa por ajustes finos (fine-tuning) para originar ferramentas e funções específicas.
- **Exemplo:** Grandes modelos de linguagem, como os desenvolvidos pela OpenAI ou Meta, que sustentam diversas aplicações de mercado.

Large Language Model (LLM)

Português: Modelo de linguagem de grande porte (substantivo masculino)

- **Definição:** Um tipo de rede neural que utiliza bilhões de conexões internas para compreender as linguagens humanas.

- **Função no Modelo:** Atua como um “motor de raciocínio” de uso geral para tarefas de linguagem natural.
- **Relação com Outros Conceitos:** A aplicação prática mais conhecida da arquitetura Transformer na área da IA generativa.
- **Exemplo:** A tecnologia fundamental que permite o funcionamento de assistentes como Claude, Gemini ou ChatGPT.

Parameter

Português: Parâmetro (substantivo masculino)

- **Definição:** Uma engrenagem interna no algoritmo que é calibrada para reduzir os erros do modelo durante o aprendizado.
- **Função no Modelo:** Utilizado para armazenar as informações e o conhecimento consolidados pela máquina.
- **Relação com Outros Conceitos:** Maior quantidade de parâmetros geralmente resulta em maior precisão e capacidade emergente.
- **Exemplo:** Um modelo com 70 bilhões de parâmetros utiliza 70 bilhões dessas “engrenagens” numéricas para calcular suas respostas.

Next-Word Prediction

Português: Predição da próxima palavra (substantivo feminino)

- **Definição:** A tarefa fundamental utilizada no treinamento de modelos de linguagem, calculando a probabilidade de cada palavra sequencial.
- **Função no Modelo:** Permite ao sistema aprender as regras básicas de gramática e a estrutura da linguagem.
- **Relação com Outros Conceitos:** A base para a maioria dos modelos autorregressivos.
- **Exemplo:** Dado o início de uma frase, o sistema calcula que a próxima palavra é muito mais provável de ser “muro” do que “oceano”.

Masked Word Prediction

Português: Predição de palavras mascaradas (substantivo feminino)

- **Definição:** Uma metodologia de treinamento onde palavras são propositalmente ocultadas dentro de uma frase, estimulando a IA a adivinhar o conteúdo ausente analisando o contexto à esquerda e à direita.
- **Função no Modelo:** Força a IA a desenvolver uma visão completa e bidirecional da estrutura das frases.
- **Relação com Outros Conceitos:** A principal abordagem de treinamento utilizada em modelos codificadores.
- **Exemplo:** Fornecer a sequência “O [MASCARADO] latiu para o carteiro” para que o algoritmo observe as palavras vizinhas e deduza que o termo oculto é “cachorro”.

Pretraining

Português: Pré-treinamento (substantivo masculino)

- **Definição:** Marca a fase inicial e mais onerosa da construção de um modelo, onde o algoritmo consome repositórios massivos da web para aprender como os humanos se comunicam.
- **Função no Modelo:** Constrói a base de conhecimento geral do mundo e a infraestrutura linguística antes de qualquer especialização.
- **Relação com Outros Conceitos:** Antecede o ajuste fino (fine-tuning) e utiliza o aprendizado auto-supervisionado como seu método principal.
- **Exemplo:** Executar clusters de supercomputadores por meses para processar portais de notícias, livros digitais e artigos da Wikipédia para que a rede aprenda a redigir.

Natural Language Processing (NLP)

Português: Processamento de linguagem natural (substantivo masculino)

- **Definição:** O subcampo da ciência da computação e da IA focado em aproximar a comunicação entre humanos e máquinas, unindo computação avançada e linguística.
- **Função no Modelo:** Capacita os computadores a ler, interpretar, analisar e gerar textos humanos da forma mais natural possível.
- **Relação com Outros Conceitos:** O campo acadêmico e tecnológico amplo que deu origem às redes neurais e aos LLMs.
- **Exemplo:** Ferramentas de acessibilidade que convertem texto em fala, plataformas de tradução instantânea ou corretores gramaticais inteligentes.

Deep Neural Network

Português: Rede neural profunda (substantivo feminino)

- **Definição:** Um modelo de software livremente inspirado no cérebro biológico, composto por múltiplas camadas digitais de processamento que trabalham juntas para resolver problemas complexos.
- **Função no Modelo:** Serve para fatiar dados complexos, com cada camada limpando, filtrando e interpretando uma parte da informação de entrada.
- **Relação com Outros Conceitos:** A infraestrutura algorítmica que viabiliza o aprendizado profundo (deep learning).
- **Exemplo:** Uma camada reconhece letras individuais, a seguinte forma palavras, a terceira capta o sentido da frase e a camada mais profunda identifica a intenção do usuário.

Temperature

Português: Temperatura (substantivo feminino)

- **Definição:** Uma configuração de controle matemático que ajusta o quão ousado, criativo ou imprevisível o modelo pode ser ao selecionar a próxima palavra.

- **Função no Modelo:** Permite alternar o comportamento da máquina entre um modo preciso e determinístico ou um modo mais solto e criativo.
- **Relação com Outros Conceitos:** Altera diretamente a distribuição de probabilidades durante a predição da próxima palavra.
- **Exemplo:** Configurar a temperatura em 0.1 para obter um relatório financeiro exato, ou subir para 0.9 para gerar ideias criativas de nomes de marcas.

Tokenization

Português: Tokenização (substantivo feminino)

- **Definição:** O processo técnico de quebrar um texto bruto em pedaços menores (chamados tokens, como palavras ou fragmentos de palavras) e convertê-los em códigos numéricos.
- **Função no Modelo:** Traduz a linguagem humana em um formato numérico que os processadores conseguem calcular.
- **Relação com Outros Conceitos:** O primeiro passo no processamento de texto, ocorrendo antes dos vetores de incorporação (embeddings) e dos mecanismos de autoatenção.
- **Exemplo:** Divisão da palavra “Inacreditável” em fragmentos menores [“In”, “acreditá”, “vel”] e repasse da sequência numérica correspondente para a máquina.

Capítulo 2: Tokenização e Embeddings

Token

Tradução: Token.

O que é: Cada um dos pedaços de texto gerados pelo tokenizador (uma palavra, parte dela ou um caractere especial).

Função no LLM: É a unidade discreta básica que carrega a informação linguística para dentro do modelo.

Relação com outros conceitos: É a chave para buscar os Token IDs correspondentes na tabela do Vocabulário.

Exemplo do Livro

```
tokens_livro = [  
    'I', 'HAD', 'always', 'thought', 'Jack', 'Gisburn', 'rather',  
    'a', 'cheap', 'genius', '-', 'though', 'a', 'good', 'fellow'  
]
```

O que o código do livro está fazendo: Aqui temos uma lista puramente textual em Python. Cada elemento (como 'I', 'always', e o travessão '-') é um token independente obtido a partir da quebra do texto original. O modelo agora enxerga a frase como uma sequência de elementos isolados.

Exemplo Criado: A palavra “correndo” dividida em dois tokens (radical + sufixo): `tokens_subpalavra = ['corr', 'endo']`.

Identificador de Token (Token ID)

Tradução: ID do Token.

O que é: Um número inteiro exclusivo associado a cada palavra/símbolo do vocabulário.

Função no LLM: Substitui o caractere textual por um número inteiro simples para que a rede neural possa usar esse número como índice de busca na memória.

Relação com outros conceitos: Representa a tradução do Token em número através do Vocabulário.

Exemplo do Livro

```
token_ids_livro = [1, 56, 2, 850, 988, 602, 533, 746, 5, 1126, 596, 5]
```

O que o código do livro está fazendo: Esta lista de inteiros substitui diretamente o texto original. Cada número aponta para o índice da palavra correspondente no dicionário do vocabulário do livro. Por exemplo, o ID 1 é o caractere " , 56 é 'It', e 2 é 's'.

Exemplo Criado: Em um sistema simplificado de controle escolar: A palavra 'presença' está no índice 5, logo: Token ID = 5.

Vocabulário (Vocabulary)

Tradução: Vocabulário.

O que é: Um dicionário (tabela de mapeamento) que contém todas as palavras que o modelo conhece e seus respectivos IDs.

Função no LLM: Funciona como o dicionário de idiomas do modelo. Define quais palavras a rede neural consegue compreender.

Relação com outros conceitos: Faz a ponte de tradução bidirecional entre Tokens e Token IDs.

Exemplo do Livro

```
vocab_livro = {
    '!': 0,
    '"': 1,
    '>': 2,
    'Her': 49,
    'Hermia': 50,
    'yourself': 1129
}
```

O que o código do livro está fazendo: Este dicionário em Python mapeia cada palavra exclusiva (chave) para um número inteiro único (valor). O livro ordena as palavras em ordem alfabética para definir a numeração: pontuações como ! e " ganham os IDs iniciais (0 e 1), palavras maiúsculas no meio e minúsculas no final.

Exemplo Criado: Um vocabulário ultra simplificado de três itens: `meu_vocab = {"o": 0, "gato": 1, "miou": 2}`.

Palavra Desconhecida (Out-of-Vocabulary — OOV)

Tradução: Fora do Vocabulário.

O que é: Um termo que não apareceu nos dados de treino e, portanto, não existe no vocabulário do modelo.

Função no LLM: Evita que o programa quebre ao ver uma palavra nova, forçando o uso de um token substituto padrão.

Relação com outros conceitos: É a falha de busca no Vocabulário, resolvida pelo BPE ou pelo uso de `<|unk|>`.

Exemplo do Livro

```
vocab_livro = {"I": 0, "thought": 1, "Jack": 2}
try:
    vocab_livro["Hello"]
except KeyError as e:
    print(f"Erro gerado: KeyError: {e}")
# Saída: Erro gerado: KeyError: 'Hello'
```

O que o código do livro está fazendo: Simula o erro que acontece quando o codificador tenta buscar a chave “Hello” no vocabulário. Como ela nunca existiu no conto de treino, o Python aciona um erro do tipo `KeyError`, travando o pipeline se não houver tratamento.

Exemplo Criado: Tentar rodar a palavra “café” em um modelo treinado apenas com textos em inglês sem suporte BPE.

Tokens Especiais (Special Tokens)

Tradução: Tokens Especiais.

O que é: Tokens artificiais inseridos para controle estrutural do modelo, como marcadores de fim de texto ou palavras desconhecidas.

Função no LLM: Instruem o modelo sobre divisões de frases, palavras inéditas ou preenchimento de lotes de forma organizada.

Relação com outros conceitos: Guardados no Vocabulário com IDs reservados.

Exemplo do Livro

```
all_tokens = sorted(list(set(preprocessed)))
all_tokens.extend(["<|endoftext|>", "<|unk|>"])
vocab = {token: integer for integer, token in enumerate(all_tokens)}
print(vocab["<|endoftext|>"]) # Saída: 1130
```

```
print(vocab["<|unk|>"]) # Saída: 1131
```

O que o código do livro está fazendo: Insere marcadores estruturais ao final do vocabulário, conferindo a eles os maiores IDs possíveis para servir como delimitadores universais de documentos e controle.

Exemplo Criado: Unir duas notícias para treino: "Notícia 1... <|endoftext|> Notícia 2...".

Byte Pair Encoding (BPE)

Tradução: Codificação por Par de Bytes (BPE).

O que é: Um algoritmo inteligente de tokenização que divide termos desconhecidos em pedaços de palavras conhecidos (subpalavras).

Função no LLM: Resolve o problema de palavras desconhecidas (OOV) decompondo-as em subunidades sem precisar recorrer a tokens de desconhecido.

Relação com outros conceitos: Técnica avançada de construção de Vocabulário utilizada no GPT-2, GPT-3 e GPT-4.

Exemplo do Livro

```
import tiktoken
tokenizer = tiktoken.get_encoding("gpt2")
integers = tokenizer.encode("Akwiw ier")
print(integers) # [33901, 86, 343, 86, 220, 959]
sub_tokens = [tokenizer.decode([i]) for i in integers]
print(sub_tokens) # ['Ak', 'w', 'ir', 'w', ' ', 'ier']
```

O que o código do livro está fazendo: Carrega o codificador BPE oficial do GPT-2 e converte a sequência desconhecida em 6 IDs inteiros, provando que o BPE quebrou a palavra em fragmentos conhecidos de subpalavras.

Exemplo Criado: Desmembrar “desatualizado” em: `tokens = ['des', 'atualizado']`.

Janela Deslizante (Sliding Window)

Tradução: Janela Deslizante.

O que é: Técnica de recortar sequências de texto movendo uma moldura de leitura ao longo da frase por um passo definido.

Função no LLM: Cria os conjuntos de dados de treinamento contínuos a partir de um texto bruto longo.

Relação com outros conceitos: O tamanho da moldura é o *Context Size* e o avanço dela é o *Stride*.

Exemplo do Livro

```
batch_1 = [367, 2885, 1464, 1807] # Frase de entrada x
batch_2 = [2885, 1464, 1807, 3619] # Segunda entrada (deslocada por 1)
```

O que o código do livro está fazendo: Demonstra a criação de amostras com passo 1: a janela remove o primeiro ID, preserva os três seguintes e inclui o próximo token do texto.

Exemplo Criado: Sentença: “IA é o futuro”. Amostra 1: “IA é” → Alvo: “é o”. Amostra 2: “é o” → Alvo: “o futuro”.

Pares de Entrada e Alvo (Input-Target Pairs)

Tradução: Pares de Entrada e Alvo.

O que é: O par de dados (X e Y) onde o modelo lê a entrada e tenta adivinhar o alvo que está deslocado de 1 posição para frente.

Função no LLM: Ensinar o modelo a prever o próximo caractere/palavra da frase.

Relação com outros conceitos: Gerados cortando o texto com a Janela Deslizante.

Exemplo do Livro

```
x = enc_sample[:4]          # [290, 4920, 2241, 287]
y = enc_sample[1:4+1]      # [4920, 2241, 287, 257]
for i in range(1, 4+1):
    contexto = x[:i]
    alvo = x[i] if i < len(x) else 257
```

O que o código do livro está fazendo: O vetor x armazena a entrada e y armazena a mesma sequência deslocada em 1 índice à frente. O laço simula como o modelo lê cumulativamente o contexto para prever o próximo token.

Exemplo Criado: Entrada: “A IA está” → Alvo: “IA está evoluindo”.

Embedding de Token (Token Embedding)

Tradução: Vetorização / Incorporação de Token.

O que é: A conversão de um ID numérico em uma linha de números decimais (vetor contínuo) que guarda o significado semântico da palavra.

Função no LLM: Permite que a rede compute relações e faça cálculos com as palavras no espaço vetorial.

Relação com outros conceitos: Executado via `torch.nn.Embedding` a partir dos Token IDs.

Exemplo do Livro

```
import torch
torch.manual_seed(123)
embedding_layer = torch.nn.Embedding(6, 3)
print(embedding_layer(torch.tensor([3])))
# tensor([[ -0.4015,  0.9666, -1.1481]])
```

O que o código do livro está fazendo: Inicializa uma matriz de pesos com 6 linhas e 3 colunas. Ao passar o índice 3, a camada atua como uma tabela de busca rápida e extrai diretamente o vetor correspondente àquele índice.

Exemplo Criado: A palavra “cachorro” convertida no vetor: $[0.12, -0.45, 0.88]$.

Embedding Posicional (Positional Embedding)

Tradução: Embedding Posicional.

O que é: Vetores adicionados à representação da palavra para identificar a ordem que ela ocupa na frase.

Função no LLM: Dá o sentido de ordem gramatical para o modelo (quem é a primeira palavra, segunda, etc.).

Relação com outros conceitos: É somado elemento a elemento ao Token Embedding.

Exemplo do Livro

```
context_length = 4
output_dim = 256
pos_embedding_layer = torch.nn.Embedding(context_length, output_dim)
pos_embeddings = pos_embedding_layer(torch.arange(context_length))
print(pos_embeddings.shape) # torch.Size([4, 256])
```

O que o código do livro está fazendo: Cria uma sequência $[0, 1, 2, 3]$ e extrai 4 vetores posicionais de 256 dimensões cada. Esses vetores serão somados aos embeddings dos tokens para embutir informação posicional.

Exemplo Criado: Frase: “Eu vou”. Vetor final de ‘vou’ = `embedding(‘vou’) + pos_embedding(índice 1)`.

DataLoader

Tradução: Carregador de Dados.

O que é: Um componente que agrupa amostras de dados individuais em blocos (lotes ou batches) de tamanho igual para processamento rápido.

Função no LLM: Facilita a alimentação paralela dos dados de treinamento na memória da placa de vídeo (GPU).

Relação com outros conceitos: Agrupa e itera sobre os dados do Dataset.

Exemplo do Livro

```
dataloader = create_dataloader_v1(  
    raw_text, batch_size=8, max_length=4, stride=4, shuffle=False  
)  
inputs, targets = next(iter(dataloader))  
print(inputs.shape) # torch.Size([8, 4])
```

O que o código do livro está fazendo: Converte os dados textuais em lotes estruturados como matrizes de 8 linhas por 4 colunas prontas para serem processadas em lote pela GPU.

Exemplo Criado: Agrupar alunos de 2 em 2: Lote 1: [Aluno A, Aluno B], Lote 2: [Aluno C, Aluno D].

Pré-treinamento (Pretraining)

Tradução: Pré-treinamento.

O que é: A fase em que o LLM lê textos em larga escala de forma autônoma para aprender regras gramaticais e estruturas linguísticas fundamentais.

Função no LLM: Cria um modelo genérico básico de compreensão de linguagem (Foundation Model).

Relação com outros conceitos: Calibra os valores iniciais da Matriz de Pesos de Embedding e dos blocos Transformer.

Exemplo do Livro

```
# Etapa 1: Pre-treinamento (Next-token prediction em dados sem rotulos)  
# --> Gera o Foundation Model (Modelo base)  
# Etapa 2: Fine-tuning (Aprender a classificar ou responder tarefas)  
# Etapa 3: Alinhamento (Aprender a seguir regras de assistente pessoal)
```

O que o código/diagrama do livro está fazendo: Demonstra que o pipeline de dados implementado no capítulo atende à Etapa 1, ensinando o modelo a prever sequências antes de qualquer especialização.

Exemplo Criado: Input: “O sol brilha no...” → Alvo: “céu”. A rede ajusta pesos para minimizar o erro.

Predição da Próxima Palavra (Next-Word Prediction Task)

Tradução: Tarefa de Predição da Próxima Palavra.

O que é: O objetivo de aprendizado do modelo: tentar adivinhar a próxima palavra dada uma sequência anterior de contexto.

Função no LLM: É o mecanismo que obriga o modelo a aprender semântica e sintaxe tentando diminuir a taxa de erro.

Relação com outros conceitos: O modelo tenta prever os elementos contidos no tensor de Alvo (Target).

Exemplo do Livro

```
[LLMs] -----> target: learn
[LLMs, learn] -----> target: to
[LLMs, learn, to] -----> target: predict
[LLMs, learn, to, predict] -----> target: one
```

O que o código do livro está fazendo: Exibe a expansão progressiva do contexto. A rede calcula perdas em cada passo para ajustar suas conexões internas.

Exemplo Criado: Digitar “Estou com muito” no celular e o teclado sugerir “sono” ou “fome”.

Word2Vec

Tradução: Word2Vec.

O que é: Um modelo clássico de embedding estático onde cada palavra tem um vetor fixo atribuído na biblioteca de termos.

Função no LLM: Mostrou que é possível agrupar termos parecidos matematicamente através de distâncias vetoriais.

Relação com outros conceitos: É a versão estática e não contextualizada de um Token Embedding.

Exemplo do Livro

```
dist_aves = torch.dist(vetor("eagle"), vetor("duck")) # Distancia curta (ex: 0.2)
dist_cidade = torch.dist(vetor("eagle"), vetor("Berlin")) # Distancia longa (ex: 1.5)
```

O que o código do livro está fazendo: Ilustra a proximidade euclidiana entre conceitos semanticamente irmãos (pássaros) contra conceitos não correlacionados (pássaro e cidade).

Exemplo Criado: No espaço RGB, vermelho e rosa compartilham coordenadas próximas, enquanto verde fica distante.

Espaço de Alta Dimensionalidade (High-Dimensional Space)

Tradução: Espaço de Alta Dimensionalidade.

O que é: A estrutura geométrica com centenas ou milhares de dimensões onde os vetores de embedding estão posicionados.

Função no LLM: Permite expressar relações muito ricas entre os dados (tempo verbal, gênero, pluralidade, emoção, categoria).

Relação com outros conceitos: Define a largura da coluna dos Token Embeddings (d_{in} , d_{out}).

Exemplo do Livro

```
GPT2_embedding_dim = 768
GPT3_embedding_dim = 12288
```

O que o código do livro está fazendo: Define a quantidade de eixos coordenados usados por arquiteturas reais de LLMs: 768 eixos no GPT-2 padrão e 12.288 eixos no GPT-3.

Exemplo Criado: Catalogar livros considerando simultaneamente 200 critérios distintos (gênero, páginas, ano, país, tom).

Embeddings Posicionais Absolutos vs. Relativos

Tradução: Embeddings Posicionais Absolutos vs. Relativos.

O que é: Métodos para informar ao modelo em qual posição a palavra se encontra na frase.

Função no LLM: Evita que a frase seja interpretada de forma desordenada (como um saco de palavras soltas).

Relação com outros conceitos: Variantes da técnica de Positional Embedding.

Exemplo do Livro

```
posicao_da_palavra = 0 # 1a palavra da frase
vetor_posicao_0 = pos_embedding_layer(torch.tensor([posicao_da_palavra]))
```

O que o código do livro está fazendo: Demonstra a abordagem absoluta adotada no GPT, onde cada posição física fixa da frase (índice 0, 1, 2) possui seu próprio vetor dedicado.

Exemplo Criado: Absoluto: “Seu armário é o número 3”. Relativo: “Seu armário está duas portas à esquerda da recepção”.

Codificação One-Hot (One-Hot Encoding)

Tradução: Codificação One-Hot.

O que é: Representar uma palavra por uma lista binária cheia de zeros, contendo apenas um número 1 indicando a sua posição na lista.

Função no LLM: É o equivalente matemático teórico à camada de embeddings. Como multiplicar matrizes por vetores cheios de zeros é ineficiente, usamos o `nn.Embedding` que atua como busca direta de linha.

Relação com outros conceitos: A técnica pesada que os Token Embeddings substituem por baixo do capô.

Exemplo do Livro

```
one_hot = torch.tensor([0.0, 0.0, 0.0, 1.0, 0.0, 0.0]) # ID 3
pesos = embedding_layer.weight
vetor_resultado = one_hot @ pesos
# Retorna exatamente a linha 3 dos pesos
```

O que o código do livro está fazendo: Prova matematicamente que extrair uma linha de pesos de forma indexada é idêntico a realizar uma multiplicação matricial completa por um vetor One-Hot.

Exemplo Criado: Um painel com 4 botões de lâmpadas onde apenas um único botão de cada vez pode ser pressionado.

Matriz de Pesos de Embedding (Embedding Weight Matrix)

Tradução: Matriz de Pesos de Embedding.

O que é: A tabela interna contendo os parâmetros da camada de embeddings com formato [Vocab Size, Embedding Dim].

Função no LLM: Armazena os vetores contínuos modificáveis de cada uma das palavras durante o aprendizado do modelo.

Relação com outros conceitos: É a estrutura consultada pelo ID do token para obter o Token Embedding.

Exemplo do Livro

```
print(embedding_layer.weight)
# Parameter containing:
# tensor([[ 0.3374, -0.1778, -0.1690], ...], requires_grad=True)
```

O que o código do livro está fazendo: Extrai a tabela interna de parâmetros PyTorch. A flag `requires_grad=True` garante que esses valores serão refinados pelo backpropagation a cada erro da rede.

Exemplo Criado: Uma caderneta de pintor onde cada linha contém o nome de uma tinta e as quantidades de mistura de cor primária.

Passo da Janela (Stride)

Tradução: Passo da Janela.

O que é: A quantidade de tokens que a moldura de leitura desloca-se a cada novo recorte de amostras no texto de treino.

Função no LLM: Controla a sobreposição e a repetição de dados de treino no pipeline de dados.

Relação com outros conceitos: Parâmetro central do Dataset na técnica de Sliding Window.

Exemplo do Livro

```
# Se tokens: [0, 1, 2, 3, 4, 5, 6, 7], max_length=4 e stride=4:
# Lote 1: [0, 1, 2, 3] -> (Pula 4 tokens) -> Lote 2: [4, 5, 6, 7]
```

O que o código do livro está fazendo: Demonstra que igualar o passo ao tamanho da janela evita sobreposições entre trechos consecutivos, acelerando o treinamento e reduzindo redundâncias.

Exemplo Criado: Uma pessoa caminhando: dar passos curtos de 20cm com alta sobreposição versus passadas largas de 70cm.

Transmissão de Tensores (Broadcasting)

Tradução: Transmissão (Broadcasting) de Tensores.

O que é: Um recurso matemático que replica virtualmente as dimensões de um tensor menor para que ele possa ser operado com um maior.

Função no LLM: Permite somar o mesmo vetor de posições [Context Length, Embedding Dim] a todas as frases do lote [Batch Size, Context Length, Embedding Dim] sem gastar memória duplicando dados.

Relação com outros conceitos: Permite unir eficientemente o Token Embedding com o Positional Embedding.

Exemplo do Livro

```
# token_embeddings.shape -> torch.Size([8, 4, 256])
# pos_embeddings.shape   -> torch.Size([4, 256])
input_embeddings = token_embeddings + pos_embeddings
print(input_embeddings.shape) # torch.Size([8, 4, 256])
```

O que o código do livro está fazendo: Soma um tensor 2D a um tensor 3D. O PyTorch estica virtualmente o tensor de posições 8 vezes, realizando a soma elemento a elemento sem alocar memória extra.

Exemplo Criado: Aplicar um único carimbo de cabeçalho em 50 folhas de contrato dispostas em uma mesa.

Capítulo 3: Mecanismos de Atenção (Coding Attention Mechanisms)

Gargalo do Encoder-Decoder em RNNs (RNN Bottleneck)

Tradução: Gargalo de Sequências Longas em Redes Recorrentes.

Conceito: Nas redes neurais recorrentes tradicionais (RNNs), o processamento opera de forma estritamente sequencial: o modelo processa token por token e atualiza iterativamente uma única célula de memória, denominada estado oculto (*hidden state*). Ao final da sequência, esse único vetor numérico de dimensão fixa é forçado a reter o significado de toda a sentença. Em sentenças longas, esse afunilamento causa a perda inevitável de contexto dos tokens iniciais e atenua a propagação de gradientes, inviabilizando a captura de dependências sintáticas de longo alcance.

Função no LLM: Constitui a principal motivação histórica para o desenvolvimento do mecanismo de atenção e da arquitetura Transformer, estabelecendo a necessidade de acesso aleatório e direto à totalidade da memória de contexto.

Relação com outros conceitos: Originou a formulação da Atenção de Bahdanau (2014) e levou à substituição das camadas recorrentes pela autoatenção em modelos de linguagem modernos.

Exemplo do Livro

```
# Representacao esquematica do Encoder-Decoder RNN (Figura 3.4 do livro):
# Encoder sequencial: [Kannst] -> [du] -> [mir] -> [helfen]
# Compressao extrema:
#                               |
#                               [Estado Oculto Final]
# Decoder sequencial:
#                               |
#                               [Can] <- [you] <- [help]
```

O que o código do livro está fazendo: O diagrama ilustra o ponto de afunilamento no qual o codificador comprime todas as entradas em um único vetor intermediário. O decodificador opera sem comunicação direta com as posições anteriores da sequência, dependendo exclusivamente da capacidade finita de representação desse estado final.

Exemplo Criado: Comprimir um documento técnico extenso em um único resumo condensado antes de utilizá-lo para responder a consultas analíticas complexas.

Atenção de Bahdanau (Bahdanau Attention Mechanism)

Tradução: Mecanismo de Atenção Aditiva de Bahdanau.

Conceito: Metodologia introduzida por Dzmitry Bahdanau em 2014 que modificou a dependência sequencial do estado oculto fixo. Em vez de restringir o decodificador unicamente ao estado final, o mecanismo estabelece canais de conexão dinâmicos com todos os estados intermediários gerados pelo codificador. A cada novo token predito, calcula-se a relevância relativa de cada palavra de entrada, atribuindo coeficientes proporcionais de ponderação para orientar a geração.

Função no LLM: Serviu como prova de conceito fundamental de que a similaridade vetorial resolve a perda de contexto em sequências longas, inspirando o surgimento da arquitetura Transformer.

Relação com outros conceitos: Antecessor histórico direto da Autoatenção (*Self-Attention*). Distingue-se por computar o alinhamento entre duas sequências distintas (tradução interlinguística), enquanto a autoatenção opera internamente sobre os elementos de uma mesma sequência.

Exemplo do Livro

```
# Tradução com atencao seletiva (Figura 3.5 do livro):
# Para gerar a palavra "help", o decodificador avalia todos os inputs:
# [Kannst] -> Peso baixo (linha fina)
# [du]      -> Peso baixo (linha fina)
```

```
# [mir] -> Peso intermediario
# [helfen] -> PESO ELEVADO (linha espessa: conexao direta identificada)
```

O que o código do livro está fazendo: O diagrama demonstra que a geração de cada termo no decodificador consulta ativamente todos os estados de entrada. As conexões com maior espessura representam coeficientes de atenção elevados, direcionando o fluxo de processamento para o termo correspondente no idioma de origem.

Exemplo Criado: Consultar seções específicas de um manual de engenharia durante a redação de cada etapa de um procedimento técnico, em vez de depender apenas da memorização preliminar de todo o texto.

Autoatenção (Self-Attention)

Tradução: Mecanismo de Autoatenção.

Conceito: Mecanismo central da arquitetura Transformer no qual os tokens pertencentes a uma mesma sequência calculam correlações e afinidades mútuas entre si. Esse processo permite que cada palavra recalibre sua representação vetorial em função dos termos adjacentes e distantes na frase, viabilizando a desambiguação contextual sem a necessidade de processamento temporal recorrente.

Função no LLM: Permite o processamento totalmente paralelizado de sequências textuais em aceleradores gráficos (GPUs), construindo representações semânticas contextuais ricas em que termos polissêmicos têm seu significado definido pelas relações com a vizinhança.

Relação com outros conceitos: Evolui para a Atenção Causal (mediante aplicação de máscara autorregressiva) e para a Atenção Multi-Head (quando paralelizada em múltiplos subespaços lineares).

Exemplo do Livro

```
# Autoatencao simplificada para a sentenca de 6 tokens:
# 1. Pontuacoes de similaridade entre todos os pares:
attn_scores = inputs @ inputs.T
# 2. Normalizacao em distribuicao de probabilidade:
attn_weights = torch.softmax(attn_scores, dim=-1)
# 3. Geracao dos vetores contextuais enriquecidos:
all_context_vecs = attn_weights @ inputs
```

O que o código do livro está fazendo: O código executa a multiplicação da matriz de embeddings $[6, 3]$ por sua transposta $[3, 6]$, gerando a matriz quadrada de afinidades $[6, 6]$. A função Softmax normaliza essas pontuações em distribuições de probabilidade, e a multiplicação matricial final '@ inputs' gera seis novos vetores de contexto.

Exemplo Criado: Em uma análise de projeto multidisciplinar, cada especificação de subsistema é confrontada com todos os demais requisitos técnicos para ajustar os parâmetros globais da estrutura.

Vetor de Contexto (Context Vector — z)

Tradução: Vetor de Contexto Enriquecido.

Conceito: Representação vetorial gerada como saída da camada de atenção, superando a representação estática do embedding de entrada original. O vetor z é obtido a partir de uma combinação linear (média ponderada) de todos os vetores da sequência, calibrada pelos pesos de atenção normalizados, refletindo a semântica contextualizada do token na oração.

Função no LLM: Constitui o sinal de saída que alimenta as camadas subsequentes da rede neural, garantindo que o modelo opere sobre tensores dinâmicos e sensíveis ao contexto.

Relação com outros conceitos: Expresso formalmente pela soma ponderada $z^{(i)} = \sum_j \alpha_{ij} x^{(j)}$ na formulação básica, e por $z^{(i)} = \sum_j \alpha_{ij} v^{(j)}$ na formulação com matrizes de projeção de Valores (V).

Exemplo do Livro

```
query = inputs[1] # Segundo token da frase: "journey"
context_vec_2 = torch.zeros(query.shape)
for i, x_i in enumerate(inputs):
    context_vec_2 += attn_weights_2[i] * x_i
print(context_vec_2)
# Saída do livro: tensor([0.4419, 0.6515, 0.5683])
```

O que o código do livro está fazendo: O trecho isola o segundo token e acumula as contribuições de todos os vetores de entrada multiplicadas pelos seus respectivos coeficientes escalares $\alpha_{2,i}$. O vetor tridimensional resultante expressa a palavra “journey” modulada pelo contexto da oração.

Exemplo Criado: Uma solução química onde uma substância base recebe frações calibradas de reagentes secundários, resultando em um composto com propriedades químicas ajustadas ao meio reativo.

Pontuações de Atenção (Attention Scores — ω)

Tradução: Pontuações de Atenção Brutas (Não Normalizadas).

Conceito: Grandezas escalares preliminares de afinidade mútua entre pares de tokens, calculadas por meio do produto escalar entre seus respectivos vetores de representação. Quantificam a intensidade geométrica de alinhamento no espaço latente sem restrições probabilísticas, podendo assumir valores positivos, nulos ou negativos antes da conversão pelo Softmax.

Função no LLM: Quantifica numericamente a força do alinhamento geométrico entre posições da sequência antes do reescalonamento em distribuições probabilísticas.

Relação com outros conceitos: Constituem o argumento de entrada direto da função Softmax e derivam da operação matricial $\Omega = QK^T$ em formulações com projeções lineares.

Exemplo do Livro

```
query = inputs[1] # Token "journey"
attn_scores_2 = torch.empty(inputs.shape[0])
for i, x_i in enumerate(inputs):
    attn_scores_2[i] = torch.dot(x_i, query)
print(attn_scores_2)
# Saída: tensor([0.9544, 1.4950, 1.4754, 0.8434, 0.7070, 1.0865])
```

O que o código do livro está fazendo: O laço itera pela sequência e calcula o produto escalar entre o segundo token e as demais palavras. O valor máximo de 1.4950 ocorre na autoconsulta, seguido por 1.4754 com a palavra “starts”, indicando proximidade semântica.

Exemplo Criado: A apuração de pontuações brutas de correlação estatística entre variáveis antes da normalização em coeficientes padronizados entre 0 e 1.

Similaridade por Produto Escalar (Dot Product Similarity)

Tradução: Produto Escalar como Métrica de Similaridade.

Conceito: Operação de álgebra linear que multiplica os termos homólogos de dois vetores e soma seus produtos: $a \cdot b = \sum_k a_k b_k$. Geometricamente, vetores alinhados no mesmo sentido produzem grandezas positivas elevadas, vetores ortogonais resultam em zero, e vetores em sentidos opostos assumem valores negativos.

Função no LLM: Atua como a métrica preferencial para cômputo de afinidade semântica devido à extrema eficiência de processamento sob multiplicações matriciais massivas em GPUs e TPUs.

Relação com outros conceitos: Fundamenta a Atenção Produto-Escalar Escalonada e todas as etapas de projeção e emparelhamento de tensores.

Exemplo do Livro

```
res = 0.
for idx, element in enumerate(inputs[0]):
    res += inputs[0][idx] * query[idx]
print(res) # tensor(0.9544)
print(torch.dot(inputs[0], query)) # tensor(0.9544)
```

O que o código do livro está fazendo: O trecho compara a iteração manual elemento a elemento com a função otimizada ‘torch.dot’, demonstrando a equivalência algébrica e a simplicidade subjacente ao cálculo de similaridade por produto escalar.

Exemplo Criado: A composição de forças vetoriais concorrentes: forças atuando na mesma direção maximizam o trabalho resultante; forças perpendiculares não exercem trabalho conjunto.

Pesos de Atenção (Attention Weights — α)

Tradução: Pesos de Atenção Normalizados.

Conceito: Coeficientes escalares obtidos pela aplicação da função Softmax sobre as pontuações brutas de atenção (ω). Atendem às propriedades formais de uma distribuição de probabilidade: são estritamente não negativos e a soma dos elementos de cada linha totaliza exatamente 1.0 (100%). Expressam a proporção exata de foco atribuída a cada token da oração.

Função no LLM: Normaliza e estabiliza a escala de ativações na rede neural, garantindo que seqüências com diferentes extensões de contexto mantenham magnitude numérica estável.

Relação com outros conceitos: São os multiplicadores que ponderam os vetores de Valores (V) para gerar os vetores de Contexto (Z), expressos por $\alpha = \text{softmax}(\omega)$.

Exemplo do Livro

```
attn_weights = torch.softmax(attn_scores, dim=-1)
print(attn_weights)
print("Soma das linhas:", attn_weights.sum(dim=-1))
# Saída: tensor([1.0000, 1.0000, 1.0000, 1.0000, 1.0000, 1.0000])
```

O que o código do livro está fazendo: O comando aplica a normalização Softmax ao longo da última dimensão (‘dim=-1’). A operação ‘.sum(dim=-1)’ valida que todas as linhas somam rigorosamente 1.0, constituindo distribuições probabilísticas formais.

Exemplo Criado: A distribuição percentual de uma cota financeira global entre múltiplos centros de custo de um laboratório, onde a soma dos repasses deve totalizar impreterivelmente 100%.

Atenção Produto-Escalar Escalonada (Scaled Dot-Product Attention)

Tradução: Atenção Produto-Escalar Dimensionada por $\sqrt{d_k}$.

Conceito: Calibração analítica na qual as pontuações de produto escalar são divididas por $\sqrt{d_k}$ (raiz quadrada da dimensão das chaves). Em dimensões elevadas ($d_k \geq 64$), o produto escalar atinge valores de grande magnitude, deslocando o operador Softmax para regiões de saturação assintótica onde as derivadas tendem a zero. A divisão por $\sqrt{d_k}$ preserva a variância unitária dos sinais e mitiga o desvanecimento de gradientes (*vanishing gradients*).

Função no LLM: Garante estabilidade analítica indispensável para que o algoritmo de retropropagação atualize de forma convergente arquiteturas profundas com bilhões de conexões.

Relação com outros conceitos: Equação clássica proposta por Vaswani et al. (2017) e adotada em todas as variações modernas da arquitetura Transformer.

Exemplo do Livro

```
d_k = keys.shape[-1]
# Divisao estabilizadora por raiz quadrada de d_k:
attn_weights_2 = torch.softmax(attn_scores_2 / d_k**0.5, dim=-1)
print(attn_weights_2)
# Saída: tensor([0.1500, 0.2264, 0.2199, 0.1311, 0.0906, 0.1820])
```

O que o código do livro está fazendo: O código afere a dimensão dos vetores de chaves ($d_k = 2$) e divide o tensor de pontuações por $2^{0.5} \approx 1.414$ antes do Softmax, prevenindo a saturação em distribuições degeneradas do tipo de grau.

Exemplo Criado: O emprego de um atenuador de sinal em um circuito de instrumentação para evitar que tensões elevadas saturam a escala de um conversor analógico-digital.

Matrizes de Pesos Treináveis de Projeção (W_q, W_k, W_v)

Tradução: Matrizes de Pesos Treináveis de Projeção.

Conceito: Conjunto de tensores com parâmetros ajustáveis que transformam os embeddings originais em três subespaços funcionais: Consulta (W_q), Chave (W_k) e Valor (W_v). Durante a retropropagação, os coeficientes dessas matrizes são refinados para projetar cada palavra em representações especializadas para busca, indexação e extração de conteúdo semântico.

Função no LLM: Confere plasticidade paramétrica e capacidade de aprendizado à camada de atenção, viabilizando a extração de padrões abstratos a partir de grandes volumes de texto.

Relação com outros conceitos: São tensores registrados no dicionário de estados (`state_dict`) do modelo, otimizados por algoritmos como o AdamW.

Exemplo do Livro

```
torch.manual_seed(123)
# Inicializacao manual das matrizes de projecao [d_in, d_out]:
W_query = torch.nn.Parameter(torch.rand(d_in, d_out), requires_grad=False)
W_key    = torch.nn.Parameter(torch.rand(d_in, d_out), requires_grad=False)
W_value  = torch.nn.Parameter(torch.rand(d_in, d_out), requires_grad=False)
```

O que o código do livro está fazendo: O trecho instancia as matrizes de projeção linear com formato $[d_{in} = 3, d_{out} = 2]$. O parâmetro `requires_grad = False` empregado em *carter didticopreliminarparasimplif*

Exemplo Criado: A passagem de um mesmo feixe de luz por três prismas ópticos distintos, gerando espectros luminosos específicos para diferentes modalidades de análise física.

Consulta, Chave e Valor (Query, Key, Value — Q, K, V)

Tradução: Consulta (Q), Chave (K) e Valor (V).

Conceito: Decomposição funcional baseada no paradigma de recuperação de informação de bancos de dados:

- **Query (Q):** Vetor de consulta emitido pelo token atual para sondar a relação com os demais elementos da frase.
- **Key (K):** Vetor de identificação exposto por todos os tokens para responder às consultas recebidas.
- **Value (V):** Vetor que transporta a informação semântica efetiva resgatada conforme a afinidade entre Q e K .

Função no LLM: Desacopla formalmente o critério de busca semântica da carga de conteúdo transmitida, elevando a riqueza de representação do modelo.

Relação com outros conceitos: Estruturam a equação canônica de atenção: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$.

Exemplo do Livro

```
# Projecao da Query para o token corrente:
query_2 = x_2 @ W_query
# Projecao em lote de todas as Chaves e Valores:
keys = inputs @ W_key
values = inputs @ W_value
print("keys.shape:", keys.shape)      # torch.Size([6, 2])
print("values.shape:", values.shape)  # torch.Size([6, 2])
```

O que o código do livro está fazendo: O código calcula as projeções dos dados de entrada. A matriz de 6 tokens com dimensão 3 é mapeada em tensores bidimensionais compactos de chaves e valores prontos para o cruzamento vetorial.

Exemplo Criado: Em um sistema de biblioteca digital: a Consulta é o termo de busca inserido, a Chave é o índice catalográfico do volume, e o Valor é o conteúdo das páginas do livro.

Camadas Lineares PyTorch na Atenção (nn.Linear)

Tradução: Implementação com Camadas Lineares PyTorch.

Conceito: Encapsulamento das projeções matriciais por meio do módulo nativo `nn.Linear`. Realiza a transformação linear $y = xA^T + b$ de forma vetorizada, incorporando algoritmos consagrados de inicialização estatística (como Kaiming Uniform) e permitindo desabilitar termos de viés (`bias=False`) para otimização de processamento.

Função no LLM: Padroniza os blocos da rede dentro das práticas de engenharia de software de aprendizado profundo, proporcionando estabilidade numérica e eficiência em GPU.

Relação com outros conceitos: Substitui manipulações manuais de tensores e atua como base para as classes `SelfAttention_v2`, `CausalAttention` e `MultiHeadAttention`.

Exemplo do Livro

```
class SelfAttention_v2(nn.Module):
    def __init__(self, d_in, d_out, qkv_bias=False):
        super().__init__()
        self.W_query = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_key = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_value = nn.Linear(d_in, d_out, bias=qkv_bias)

    def forward(self, x):
        keys = self.W_key(x)
        queries = self.W_query(x)
        values = self.W_value(x)
        attn_scores = queries @ keys.T
        attn_weights = torch.softmax(attn_scores / keys.shape[-1]**0.5, dim=-1)
        return attn_weights @ values
```

O que o código do livro está fazendo: A classe empacota a rotina de atenção em um módulo 'nn.Module'. O método 'forward' recebe as entradas, calcula as projeções lineares, processa a atenção escalonada e retorna os vetores de contexto sem laços iterativos manuais.

Exemplo Criado: A integração de circuitos integrados pré-fabricados em uma placa de circuito impresso em substituição a arranjos manuais de componentes discretos.

Atenção Causal / Mascarada (Causal Attention / Masked Attention)

Tradução: Mecanismo de Atenção Causal Autorregressiva.

Conceito: Restrição unidirecional imposta à matriz de atenção em modelos de linguagem geradores (como a arquitetura GPT). Durante a etapa de processamento, cada token tem permissão de calcular atenção unicamente sobre si mesmo e sobre os elementos anteriores no texto, anulando qualquer conexão com posições futuras na sequência temporal.

Função no LLM: Garante a integridade do aprendizado autorregressivo, impedindo o vazamento de informação de tokens futuros durante as etapas de treinamento e geração sequencial.

Relação com outros conceitos: Implementada algebricamente convertendo a matriz de pesos de atenção em uma estrutura triangular inferior estrita.

Exemplo do Livro

```
# Estrutura triangular da matriz de pesos causais (Figura 3.19 do livro):
# Para cada posicao temporal, os pesos futuros sao zerados:
[1.00, 0.00, 0.00, 0.00, 0.00, 0.00] # Token 1 so enxerga a si mesmo
[0.55, 0.44, 0.00, 0.00, 0.00, 0.00] # Token 2 consulta 1 e 2
[0.38, 0.30, 0.31, 0.00, 0.00, 0.00] # Token 3 consulta 1, 2 e 3
# A porcao triangular superior permanece estritamente nula
```

O que o código do livro está fazendo: O diagrama esquematiza a matriz de pesos causais, evidenciando que as posições acima da diagonal principal são estritamente nulas, bloqueando dependências com termos situados à frente na frase.

Exemplo Criado: Um leitor óptico que analisa uma fita de dados linha por linha enquanto uma cortina mecânica opaca bloqueia a visualização das linhas subsequentes.

Truque do Menos Infinito na Máscara Causal (-torch.inf + Softmax)

Tradução: Técnica de Mascaramento com Menos Infinito.

Conceito: Solução computacional analítica para imposição da máscara causal. Antes da passagem pela função Softmax, as posições superiores da matriz de pontuações (ω) são substituídas por $-\infty$ (-torch.inf). Pela definição assintótica $\lim_{x \rightarrow -\infty} e^x = 0$, o operador Softmax converte essas posições futuras automaticamente em probabilidade zero, recalculando a soma unitária da linha em um único passo vetorizado.

Função no LLM: Maximiza o desempenho computacional em hardware gráfico, eliminando divisões de renormalização adicionais e operações condicionais de corte matricial.

Relação com outros conceitos: Executado via método `.masked_fill_()` do PyTorch imediatamente antes da normalização pelo `torch.softmax`.

Exemplo do Livro

```
# 1. Mascara triangular booleana com 1s acima da diagonal principal:
mask = torch.triu(torch.ones(context_length, context_length), diagonal=1)
# 2. Preenchimento com menos infinito nas posicoes verdadeiras:
masked = attn_scores.masked_fill(mask.bool(), -torch.inf)
# 3. Softmax resolve o mascaramento e a normalizacao em passo unico:
attn_weights = torch.softmax(masked / keys.shape[-1]**0.5, dim=-1)
print(attn_weights)
```

O que o código do livro está fazendo: O comando `'torch.triu(..., diagonal=1)'` gera uma matriz booleana triangular superior. O preenchimento com `'-torch.inf'` assegura que o Softmax anule as posições futuras e equalize os pesos válidos restantes em soma unitária 1.0.

Exemplo Criado: A inserção de diodos com barreira de potencial intransponível em um ramo de circuito, tornando a corrente elétrica estritamente nula naquele caminho.

Prevenção de Vazamento de Informação (Information Leakage)

Tradução: Garantia contra Vazamento Indireto de Informação.

Conceito: Propriedade matemática assegurada pela aplicação de $-\infty$ na máscara causal. Uma vez que $e^{-\infty} = 0$, os elementos futuros são estritamente excluídos tanto do numerador quanto do denominador da função Softmax, garantindo que o resultado para os tokens válidos seja idêntico ao de uma distribuição computada sem a existência dos tokens posteriores.

Função no LLM: Assegura a validade formal do treinamento autorregressivo, impedindo qualquer transmissão indireta de contexto futuro para as ativações presentes da rede.

Relação com outros conceitos: Fundamenta a robustez teórica e a estabilidade da inferência causal na arquitetura GPT.

Exemplo do Livro

```
# Nota explicativa do livro sobre estabilidade matematica:  
# "The mathematical elegance of softmax is that despite initially including  
# all positions in the denominator, after masking with -inf, the effect of the  
# masked positions is nullified... ensuring there is no information leakage."
```

O que o código do livro está fazendo: O texto formaliza que a influência de posições futuras no denominador do Softmax é rigorosamente anulada, impedindo contaminação informacional sobre os estados precedentes.

Exemplo Criado: A blindagem eletrostática em cabos coaxiais de transmissão para impedir que sinais de alta frequência induzam correntes parasitas em canais vizinhos.

Dropout na Atenção (Attention Dropout)

Tradução: Regularização por Dropout em Pesos de Atenção.

Conceito: Técnica de regularização estocástica adaptada aos coeficientes de atenção normalizados (α). Durante a fase de treino, conexões de atenção aleatórias são forçadas a zero com probabilidade p . Para manter inalterada a magnitude média das ativações, os pesos remanescentes são multiplicados pelo fator compensatório $1/(1 - p)$ (por exemplo, escalonados por 2.0 quando $p = 0.5$).

Função no LLM: Previne o sobreajuste (*overfitting*) impedindo que o modelo crie dependências rígidas entre pares fixos de tokens, estimulando representações semânticas distribuídas e generalizáveis.

Relação com outros conceitos: Atua ativamente apenas no regime de treinamento (`model.train()`), sendo desativado automaticamente durante a inferência (`model.eval()`).

Exemplo do Livro

```
torch.manual_seed(123)  
dropout = torch.nn.Dropout(0.5) # Taxa de desativacao de 50%  
example = torch.ones(6, 6)  
print(dropout(example))  
# Saída do livro: metade das posicoes zera e os sobreviventes escalam para 2.0
```

O que o código do livro está fazendo: O trecho aplica dropout de 50% sobre uma matriz unitária. As posições ativas remanescentes são reescaladas para 2.0 ($1/0.5 = 2$), preservando a média estocástica da matriz.

Exemplo Criado: Testes de redundância em uma malha de distribuição de energia onde rotas primárias são desconectadas aleatoriamente para assegurar a operacionalidade das linhas de contingência.

Buffer Registrado no PyTorch (register_buffer)

Tradução: Registro de Buffer de Módulo PyTorch.

Conceito: Mecanismo da biblioteca PyTorch para registrar tensores que fazem parte do estado interno de um módulo, mas não constituem parâmetros treináveis (não demandam cálculo de gradientes). O método `register_buffer` vincula tensores como a máscara causal ao ciclo de vida do `nn.Module`, assegurando que sejam transferidos automaticamente entre CPU e GPU durante chamadas como `model.to(device)`.

Função no LLM: Evita falhas de hardware do tipo *RuntimeError: Expected all tensors to be on the same device* e preserva o tensor no arquivo de serialização (`state_dict`) do modelo.

Relação com outros conceitos: Utilizado no construtor `init.dasclassesCausalAttentioneMultiHeadAttention`.

Exemplo do Livro

```
self.register_buffer(
    'mask',
    torch.triu(torch.ones(context_length, context_length), diagonal=1)
)
```

O que o código do livro está fazendo: O código registra a matriz booleana triangular como buffer oficial do módulo. Caso o modelo seja enviado para aceleradores CUDA, a máscara acompanha os tensores de parâmetros de forma transparente.

Exemplo Criado: A gravação de uma tabela de constantes de calibração na memória ROM de um microcontrolador: os valores são estáticos, não sofrem escrita, mas viajam integrados ao hardware.

Classe Compacta CausalAttention

Tradução: Classe Completa de Atenção Causal para Lotes.

Conceito: Módulo integrado que unifica as projeções lineares Q, K, V , o cálculo de similaridade via transposição de lotes `transpose(1, 2)`, o mascaramento causal triangular com $-\infty$, a normalização Softmax escalonada e a camada de Dropout.

Função no LLM: Permite o processamento simultâneo de lotes tridimensionais $[B, T, C]$ de frases oriundas do DataLoader, garantindo a conformidade com a causalidade temporal autorregressiva.

Relação com outros conceitos: Constitui a unidade fundamental de processamento que serve de base para a arquitetura de Atenção Multi-Head.

Exemplo do Livro

```
class CausalAttention(nn.Module):
    def __init__(self, d_in, d_out, context_length, dropout, qkv_bias=False):
        super().__init__()
        self.d_out = d_out
        self.W_query = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_key = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_value = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.dropout = nn.Dropout(dropout)
        self.register_buffer(
            'mask',
            torch.triu(torch.ones(context_length, context_length), diagonal=1)
        )

    def forward(self, x):
        b, num_tokens, d_in = x.shape
        keys = self.W_key(x)
        queries = self.W_query(x)
        values = self.W_value(x)
        attn_scores = queries @ keys.transpose(1, 2)
        attn_scores.masked_fill_(self.mask.bool()[:num_tokens, :num_tokens],
                                -torch.inf)
        attn_weights = torch.softmax(attn_scores / keys.shape[-1]**0.5, dim=-1)
        attn_weights = self.dropout(attn_weights)
        return attn_weights @ values
```

O que o código do livro está fazendo: O método ‘`transpose(1, 2)`’ permuta os eixos de tokens e canais para executar a álgebra linear em lotes preservando a dimensão b . O corte de fatia ‘`[:num_tokens, :num_tokens]`’ ajusta a máscara dinamicamente quando o comprimento da sequência for inferior ao contexto.

Exemplo Criado: Uma linha de montagem industrial com controle de fluxo unidirecional que processa lotes contínuos de peças em paralelo sem admitir refluxo de material.

Atenção Multi-Head (Multi-Head Attention — MHA)

Tradução: Mecanismo de Atenção com Múltiplos Cabeçotes.

Conceito: Mecanismo que particiona o processamento de atenção em múltiplos subespaços lineares independentes (cabeçotes) executados em paralelo. Cada cabeça de atenção projeta os dados em representações próprias, especializando-se em diferentes aspectos da linguagem: dependências sintáticas, concordância verbal, correlações de longo alcance e referências pronominais.

Função no LLM: Eleva a capacidade representacional da arquitetura sem aumentar a carga computacional global, permitindo a extração concorrente de múltiplos padrões semânticos.

Relação com outros conceitos: No GPT-2 padrão são empregados 12 cabeçotes em paralelo com dimensão 64 ($12 \times 64 = 768$ canais totais); no GPT-3 utilizam-se 96 cabeçotes paralelos.

Exemplo do Livro

```
# Dois cabecotes independentes operando com d_out=2 (Figura 3.25 do livro):
# Cabecote 1 gera Z1 e Cabecote 2 gera Z2:
# Concatenacao ao longo da dimensao de caracteristicas (dim=-1):
# Z = torch.cat([Z1, Z2], dim=-1) -> Dimensao de saida final = 4
```

O que o código do livro está fazendo: O autor demonstra que as matrizes de contexto de cada cabeçote são concatenadas ao longo do último eixo (‘`dim=-1`’), gerando uma representação vetorial unificada que combina as saídas de todos os subespaços latentes.

Exemplo Criado: O diagnóstico simultâneo de uma máquina industrial por sensores acústicos, térmicos e de vibração, cujos sinais individuais são combinados para uma análise integrada de integridade.

Abordagem Didática por Empilhamento (MultiHeadAttentionWrapper)

Tradução: Empacotador Didático de Atenção Multi-Head.

Conceito: Montagem estrutural introdutória que instancia uma coleção de módulos `CausalAttention` encapsulados em um `nn.ModuleList`. A computação é realizada de forma iterativa por um laço em Python e os tensores de saída são concatenados com `torch.cat`. Trata-se de uma formulação conceitual intuitiva, porém computacionalmente ineficiente para execução em hardware.

Função no LLM: Serve como introdução didática ao paralelismo de cabeçotes antes do estudo da técnica avançada por divisão de pesos matriciais.

Relação com outros conceitos: É a contrapartida sequencial preliminar da versão de alto rendimento baseada em fatiamento de tensores (*weight splits*).

Exemplo do Livro

```
class MultiHeadAttentionWrapper(nn.Module):
    def __init__(self, d_in, d_out, context_length, dropout, num_heads,
                  qkv_bias=False):
        super().__init__()
        self.heads = nn.ModuleList(
            [CausalAttention(d_in, d_out, context_length, dropout, qkv_bias)
             for _ in range(num_heads)]
        )
```

```
def forward(self, x):
    return torch.cat([head(x) for head in self.heads], dim=-1)
```

O que o código do livro está fazendo: O módulo instancia ‘num_heads’ cabeotes autônomos. *Nom todo ‘forward’.*

Exemplo Criado: A operação de quatro instrumentos de medição portáteis isolados em vez do uso de um analisador multicanal integrado com barramento de alta velocidade.

Divisão Matricial de Pesos na Multi-Head (Weight Splits)

Tradução: Multi-Head Otimizada por Fatiamento de Tensores.

Conceito: Implementação de alto desempenho adotada em ambientes de produção. Em vez de instanciar blocos lineares individuais pequenos, projeta-se uma única matriz linear larga para Q , K e V . Em seguida, operações de reorganização geométrica (`.view()`) e transposição (`.transpose()`) particionam o tensor contíguo em quatro dimensões $[B, \text{num_heads}, T, \text{head_dim}]$, calculando a atenção de todos os cabeçotes em bloco na GPU.

Função no LLM: Maximiza o aproveitamento do paralelismo massivo em núcleos CUDA, suprimindo o gargalo operacional decorrente de laços sequenciais em Python.

Relação com outros conceitos: Padrão arquitetural empregado em modelos de larga escala, como GPT, LLaMA e bibliotecas industriais de aprendizado profundo.

Exemplo do Livro

```
# 1. Projecao linear unificada:
keys = self.W_key(x) # Formato original: (b, num_tokens, d_out)
# 2. Desdobramento tensorial da dimensao de saida:
keys = keys.view(b, num_tokens, self.num_heads, self.head_dim)
# 3. Permutacao de eixos para posicionar num_heads no indice 1:
keys = keys.transpose(1, 2) # Formato final: (b, num_heads, num_tokens, head_dim)
```

O que o código do livro está fazendo: O método desmembra a dimensão linear d_{out} nas variáveis ‘num_heads’ e ‘head_dim’. *A transposição move a dimensão dos cabeçotes para o eixo 1, viabilizando operações matriciais paralelas.*

Exemplo Criado: A extrusão de uma barra sólida contínua e seu corte simultâneo em 12 perfis idênticos por lâminas paralelas sincronizadas em uma única passagem industrial.

Dimensão do Cabeçote (Head Dimension — head_dim)

Tradução: Dimensão Interna do Cabeçote.

Conceito: Relação de proporcionalidade direta dada por: $\text{head_dim} = d_{\text{out}} // \text{num_heads}$. Essa formulação distribui uniformemente a largura total de características entre a quantidade de cabeçotes, garantindo que o custo computacional e o número de parâmetros de uma camada Multi-Head sejam equivalentes aos de uma atenção com cabeçote único sobre a dimensão total.

Função no LLM: Equilibra a carga de parâmetros do modelo e impõe a verificação estrutural de divisibilidade exata entre canais e cabeçotes.

Relação com outros conceitos: Exige a validação lógica no construtor da classe: `assert (d_out % num_heads == 0)`.

Exemplo do Livro

```
assert (d_out % num_heads == 0), "d_out deve ser divisível por num_heads"
self.d_out = d_out
self.num_heads = num_heads
self.head_dim = d_out // num_heads
```

O que o código do livro está fazendo: A asserção valida se a largura de saída pode ser particionada de forma inteira entre os cabeçotes configurados, definindo o tamanho do subespaço geométrico que cada cabeça gerencia.

Exemplo Criado: A repartição equitativa de uma banda de comunicação de 768 MHz entre 12 canais independentes de exatamente 64 MHz cada.

Multiplicação Matricial em Lote para Tensores 4D (Batched Matmul)

Tradução: Multiplicação Matricial em Lote em 4 Dimensões.

Conceito: Aplicação da álgebra linear sobre tensores quadridimensionais no formato $[B, \text{num_heads}, T, \text{head_dim}]$. As duas primeiras dimensões (B e num_heads) são interpretadas como identificadores de lotes independentes, ocorrendo a multiplicação matricial com o operador '@' estritamente entre as duas últimas dimensões ($[T \times \text{head_dim}]$ por $[\text{head_dim} \times T]$).

Função no LLM: Permite que as matrizes de similaridade de todas as sentenças do lote e de todos os cabeçotes de atenção sejam calculadas de forma simultânea e vetorizada.

Relação com outros conceitos: Executa com máxima eficiência a operação central de atenção: `attn_scores = queries @ keys.transpose(2, 3)`.

Exemplo do Livro

```
# Tensor 4D de teste: formato (b=1, heads=2, tokens=3, head_dim=4)
resultado = a @ a.transpose(2, 3)
print(resultado.shape) # Saída do livro: torch.Size([1, 2, 3, 3])
```

O que o código do livro está fazendo: O método '`transpose(2, 3)`' permuta os eixos de tokens e dimensões. A multiplicação matricial calcula a interação de $[3 \times 4]$ com $[4 \times 3]$, gerando matrizes de afinidade $[3 \times 3]$ para todos os cabeçotes sem laços iterativos.

Exemplo Criado: Uma prensa automatizada que estampa paralelamente matrizes de circuitos em dezenas de placas posicionadas em esteiras simultâneas.

Camada de Projeção de Saída (out_proj)

Tradução: Camada Linear de Projeção de Saída.

Conceito: Transformação linear final (`nn.Linear(d_out, d_out)`) aplicada após a concatenação das saídas de todos os cabeçotes. Como cada cabeça operou de forma isolada em seu próprio subespaço, a camada linear permite que as informações extraídas pelos diferentes cabeçotes interajam e se sintetizem mutuamente antes de seguirem para o restante da rede.

Função no LLM: Promove a fusão e a integração das representações dos múltiplos cabeçotes de atenção, homogeneizando o vetor de contexto para os blocos posteriores do Transformer.

Relação com outros conceitos: Constitui o passo final no método '`forward`' de '`MultiHeadAttention`', antecedendo as conexões residuais e a normalização de camada (`LayerNorm`).

Exemplo do Livro

```
# 1. Reagrupamento dos cabecotes e retorno para o formato 3D (b, num_tokens, d_out):
context_vec = (attn_weights @ values).transpose(1, 2)
context_vec = context_vec.contiguous().view(b, num_tokens, self.d_out)
# 2. Projecao linear integradora:
context_vec = self.out_proj(context_vec)
return context_vec
```

O que o código do livro está fazendo: O código reorganiza a disposição de memória com '`contiguous().view()`' para agrupar os cabeçotes na largura original d_{out} . A camada '`self.out_proj`' aplica a projeção linear integrada.

Exemplo Criado: Uma mesa de mixagem de áudio que recebe sinais de múltiplos microfones independentes e realiza a equalização unificada em uma faixa sonora master.

Classe Definitiva MultiHeadAttention

Tradução: Classe Consolidada de Atenção Multi-Head do GPT.

Conceito: Módulo de produção que harmoniza projeções lineares unificadas, manipulação tensorial quadridimensional, atenção causal com escalonamento por $\sqrt{d_k}$, mascaramento in-place com $-\infty$, regularização estocástica por dropout e projeção linear de saída integradora.

Função no LLM: Atua como o bloco construtivo central que é empilhado nas camadas sequenciais da arquitetura do GPT-2 e modelos afins.

Relação com outros conceitos: Substitui definitivamente com máxima performance todas as implementações preliminares de atenção formuladas nas etapas de estudo.

Exemplo do Livro

```
class MultiHeadAttention(nn.Module):
    def __init__(self, d_in, d_out, context_length, dropout, num_heads,
                  qkv_bias=False):
        super().__init__()
        assert (d_out % num_heads == 0), "d_out must be divisible by num_heads"
        self.d_out = d_out
        self.num_heads = num_heads
        self.head_dim = d_out // num_heads

        self.W_query = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_key = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_value = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.out_proj = nn.Linear(d_out, d_out)
        self.dropout = nn.Dropout(dropout)
        self.register_buffer(
            "mask",
            torch.triu(torch.ones(context_length, context_length), diagonal=1)
        )

    def forward(self, x):
        b, num_tokens, d_in = x.shape
        # Projecao e remodelagem 4D: (b, num_heads, num_tokens, head_dim)
        keys = self.W_key(x).view(b, num_tokens, self.num_heads,
                                   self.head_dim).transpose(1, 2)
        queries = self.W_query(x).view(b, num_tokens, self.num_heads,
                                        self.head_dim).transpose(1, 2)
        values = self.W_value(x).view(b, num_tokens, self.num_heads,
                                       self.head_dim).transpose(1, 2)

        # 1. Pontuacoes em paralelo:
        attn_scores = queries @ keys.transpose(2, 3)
        # 2. Mascara causal com -inf:
        mask_bool = self.mask.bool()[:num_tokens, :num_tokens]
        attn_scores.masked_fill_(mask_bool, -torch.inf)
        # 3. Softmax normalizado e escalonado:
        attn_weights = torch.softmax(attn_scores / keys.shape[-1]**0.5, dim=-1)
        attn_weights = self.dropout(attn_weights)
        # 4. Multiplicacao de valores, unificacao e projecao final:
        context_vec = (attn_weights @ values).transpose(1, 2)
        context_vec = context_vec.contiguous().view(b, num_tokens, self.d_out)
        return self.out_proj(context_vec)
```

O que o código do livro está fazendo: O módulo encapsula a engenharia da atenção multi-head em cerca de 30 linhas de código PyTorch otimizado. A partir de um lote tridimensional $[B, T, C]$, processa múltiplos cabeçotes em tensores 4D e retorna representações contextuais causais prontas

para modelos de linguagem em larga escala.

Exemplo Criado: Um módulo de propulsão integrada automotiva onde injeção eletrônica, so-brealimentação, controle de tempo de válvulas e exaustão operam simultaneamente sob uma única unidade de comando computacional.